torch.empty#

https://pytorch.org/docs/stable/generated/torch.empty.html

Description:

Returns a tensor filled with uninitialized data. The shape of the tensor is defined by the variable argument size. If torch.use_deterministic_algorithms() and torch.utils.deterministic.fill_uninitialized_memory are both set to True, the output tensor is initialized to prevent any possible nondeterministic behavior from using the data as an input to an operation. Floating point and complex tensors are filled with NaN, and integer tensors are filled with the maximum value. size (int...) – a sequence of integers defining the shape of the output tensor. Can be a variable number of arguments or a collection like a list or tuple. out (Tensor, optional) – the output tensor.

Function Signature

```
torch.empty(*size, *, out=None, dtype=None,
layout=torch.strided, device=None, requires_grad=False,
pin_memory=False, memory_format=torch.contiguous_format) →
Tensor#

Parameters

Keyword Arguments
```

Parameters

Keyword

out (Tensor, optional) – the output tensor. dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see torch.set_default_dtype()). layout (torch.layout, optional) – the desired layout of returned Tensor. Default: torch.strided. device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see torch.set_default_device()). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. requires_grad (bool, optional) – If autograd should record operations on the returned tensor. Default: False. pin_memory (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default

Code Examples

```
>>> torch.empty((2,3), dtype=torch.int64)
tensor([[ 9.4064e+13,  2.8000e+01,  9.3493e+13],
        [ 7.5751e+18,  7.1428e+18,  7.5955e+18]])
```

Notes & Warnings

NOTE Note If torch.use_deterministic_algorithms() and torch.utils.deterministic.fill_uninitialized_memory are both set to True, the output tensor is initialized to prevent any possible nondeterministic behavior from using the data as an input to an operation. Floating point and complex tensors are filled with NaN, and integer tensors are filled with the maximum value.

Detailed Documentation

Documentation

Returns a tensor filled with uninitialized data. The shape of the tensor is defined by the variable argument size.

If torch.use_deterministic_algorithms() and torch.utils.deterministic.fill_uninitialized_memory are both set to True, the output tensor

is initialized to prevent any possible nondeterministic behavior from using the data as an input to an operation. Floating point and complex tensors are filled with NaN, and integer tensors are filled with the maximum value.

size (int...) – a sequence of integers defining the shape of the output tensor. Can be a variable number of arguments or a collection like a list or tuple.

out (Tensor, optional) – the output tensor.

dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see torch.set_default_dtype()).

layout (torch.layout, optional) – the desired layout of returned Tensor. Default: torch.strided.

device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see torch.set_default_device()). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

requires_grad (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

pin_memory (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False.

memory_format (torch.memory_format, optional) – the desired memory format of returned Tensor. Default: torch.contiguous_format.